



María Colio Tresgalo

Darío Briongos García

Raúl Calderón Gómez

Jino Olivera Rudas

MEMORIA TÉCNICA

PROYECTO INTERMODULAR – TASKFLOW

1. Introducción

TaskFlow es una aplicación web de gestión de proyectos y tareas basada en metodología Kanban. El objetivo del sistema es permitir la organización de equipos, proyectos y tareas mediante una interfaz moderna, intuitiva y segura.

El desarrollo se ha realizado bajo una arquitectura desacoplada, utilizando React en el frontend y Laravel como backend API REST. La autenticación se ha implementado mediante Laravel Sanctum.

Esta memoria técnica se centra especialmente en la arquitectura y diseño del frontend, conforme a las indicaciones del proyecto. Asimismo, se detallan las decisiones técnicas adoptadas, el sistema de seguridad implementado y el proceso de despliegue en entorno cloud.

Se utilizó Vite como herramienta de desarrollo y construcción del frontend debido a su rapidez en el entorno de desarrollo y su sistema de empaquetado optimizado para producción. Vite ofrece tiempos de compilación significativamente menores en comparación con herramientas tradicionales como Webpack, lo que mejora la eficiencia durante el desarrollo.

2. Arquitectura del Frontend

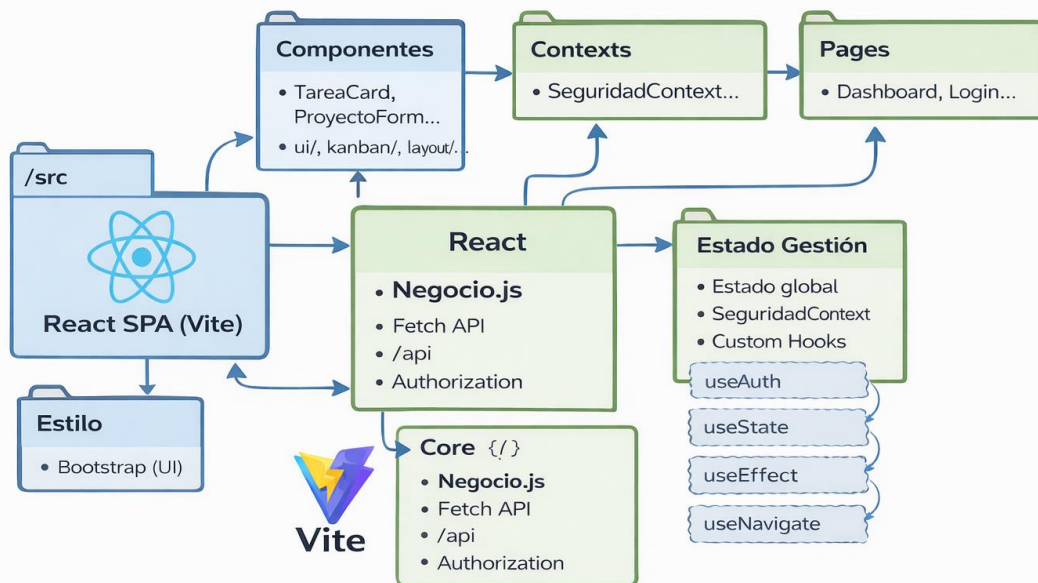
2.1 Enfoque arquitectónico

El frontend se ha desarrollado como una SPA (Single Page Application) utilizando React. Esta arquitectura permite:

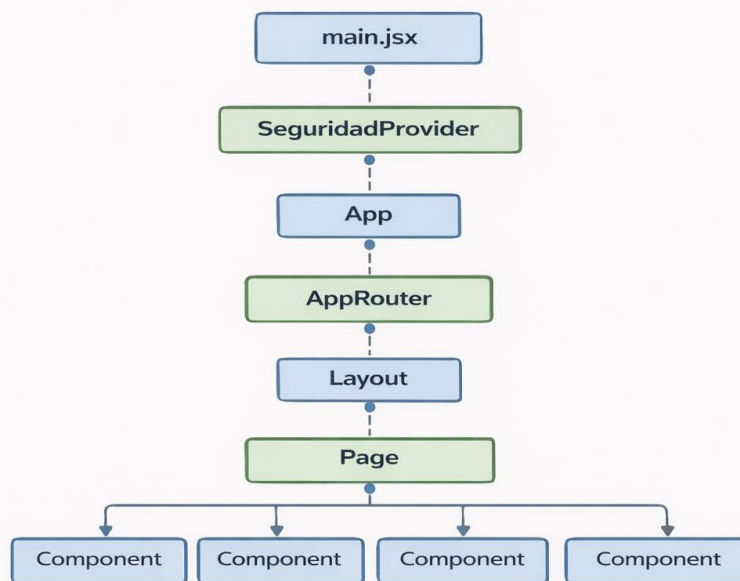
- Navegación sin recarga de página.
- Mayor fluidez en la experiencia de usuario.
- Separación clara entre interfaz y lógica de negocio.

Se ha aplicado un enfoque modular por dominios funcionales (tareas, proyectos, equipos, usuarios), facilitando la mantenibilidad y escalabilidad del sistema.

Arquitectura Global del Frontend



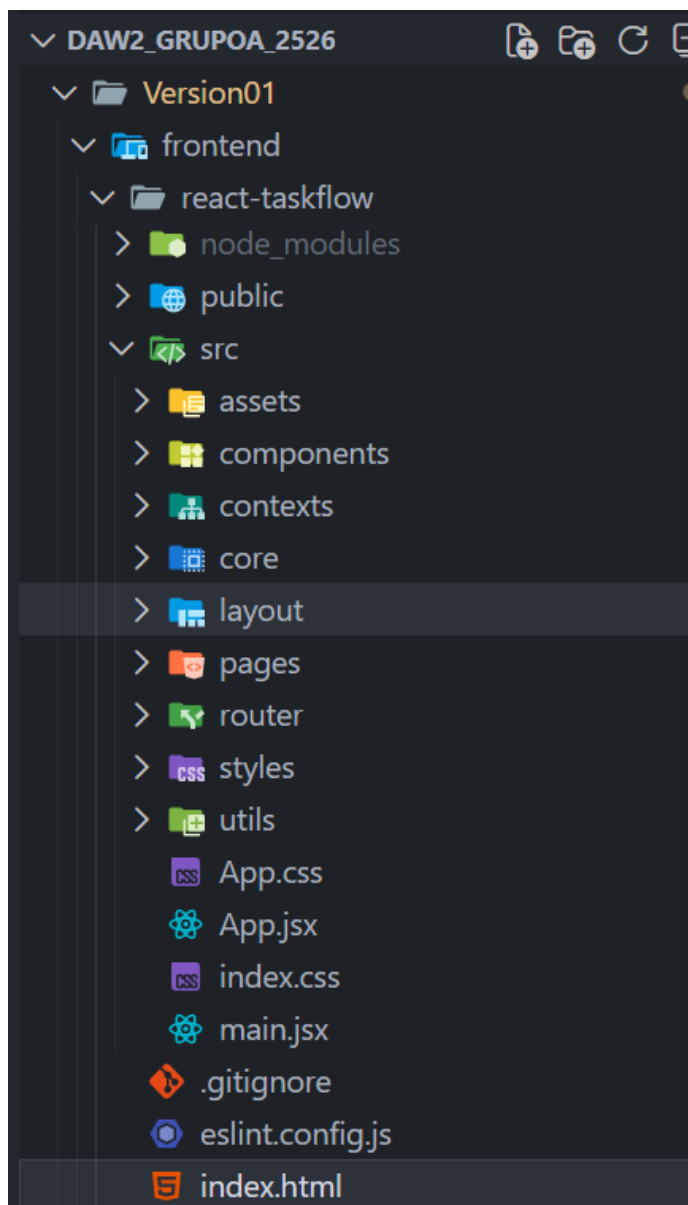
Jerarquía de Componentes en TaskFlow



2.2 Estructura de Carpetas

La estructura sigue el esquema generado por Vite como herramienta de bundling, adaptado posteriormente a una organización modular por dominios.

La estructura real del proyecto es la siguiente:



Explicación

- components/ contiene componentes reutilizables organizados por dominio.
- pages/ agrupa vistas completas asociadas a rutas.
- contexts/ gestiona el estado global (autenticación).
- core/ centraliza las llamadas a la API.
- router/ define el sistema de navegación.
- styles/ y assets/ contienen recursos visuales.

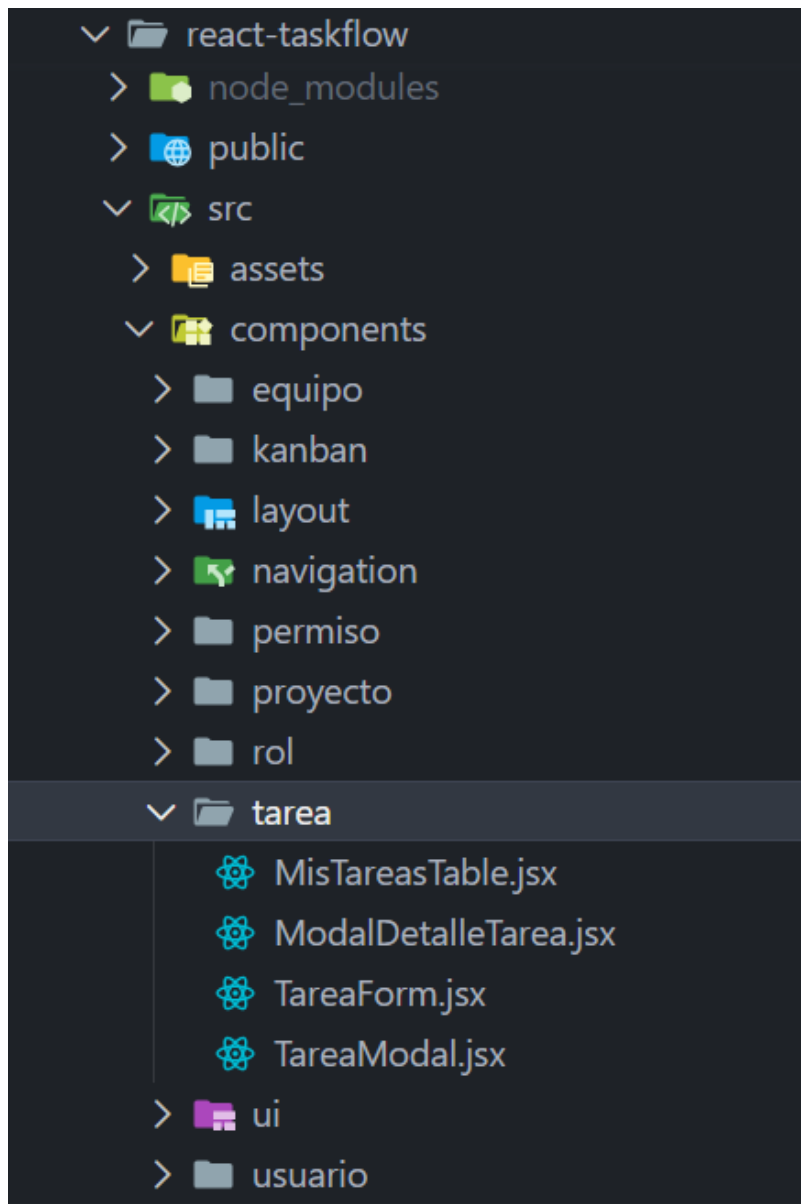
Esta organización permite una clara separación entre lógica visual, lógica de negocio y navegación.

2.3 Organización de Componentes

Los componentes se dividen en:

- Componentes de presentación (UI).
- Componentes de dominio (TareaCard, ProyectoForm).
- Componentes de layout (Sidebar, Header).
- Componentes funcionales del tablero Kanban.

Se ha aplicado el principio de responsabilidad única (SRP), donde cada componente tiene una función específica. Esto facilita la reutilización y reduce el acoplamiento.



2.4 Gestión de Estado

La gestión del estado global se realiza mediante:

Context API

Se ha creado SeguridadContext para:

- Usuario autenticado.
- Token de acceso.
- Equipo activo.
- Rol activo.
- Permisos.

El contexto se define mediante createContext() y es expuesto junto a su componente proveedor (SeguridadProvider), el cual envuelve la aplicación para distribuir el estado global.

```
import AppEnrutador from "../router/AppRouter";
createRoot(document.getElementById("root")).render(
  <StrictMode>
    { /*
  <App /> */}
    <SeguridadProvider>
      <AppEnrutador />
    </SeguridadProvider>
  </StrictMode>,
);
```

El uso de Context API permite evitar el denominado “prop drilling”, que consiste en pasar propiedades innecesarias a través de múltiples niveles de componentes. Mediante Context se centraliza la información de sesión (usuario autenticado, token y permisos), permitiendo que cualquier componente acceda directamente a estos datos sin generar dependencias intermedias.

```

1  import { useState, createContext, useEffect } from "react";
2  import Negocio from "../core/Negocio";
3
4  const SeguridadContext = createContext();
5
6  function SeguridadProvider({ children }) {
7      const [datos, setDatos] = useState({
8          usuario: null,
9          token: null,
10         equipos: [],
11         equipoActivo: null,
12         rolActivo: null,
13         tienePermisos: false,
14     });
15
16     const [cargando, setCargando] = useState(true);
17
18     // Restaurar sesión al montar la app
19     useEffect(() => {
20         try {
21             const token = localStorage.getItem("token");
22             const usuario = localStorage.getItem("usuario");
23             const equipos = localStorage.getItem("equipos");
24             const equipoActivo = localStorage.getItem("equipoActivo");

```

Hooks utilizados

- useState para gestión de estados locales.
- useEffect para efectos secundarios y llamadas API.
- useContext para acceso a estado global.
- useNavigate para redirección programática.

Se valoró la utilización de Redux como gestor de estado global; sin embargo, se optó por Context API al ser suficiente para la complejidad funcional del proyecto. La incorporación de Redux hubiera añadido una capa adicional de abstracción innecesaria, generando mayor complejidad estructural sin aportar beneficios proporcionales. Esta decisión permitió mantener el sistema alineado con los objetivos académicos, aplicando una solución técnica adecuada sin incurrir en sobreingeniería.

2.5 Sistema de Enrutado

El enrutado se implementa mediante React Router DOM v6.

Se distinguen:

- Rutas públicas (Login).
- Rutas protegidas (Dashboard, Tableros).
- Layout compartido (Sidebar + Header).

Las rutas protegidas implementadas mediante React Router validan la existencia del token de autenticación antes de permitir el renderizado de vistas privadas. En caso de no detectarse sesión activa, el sistema redirige automáticamente al usuario al formulario de login. Esta estrategia añade una capa de control en el frontend, mejorando la experiencia de seguridad al impedir el acceso visual a recursos protegidos, complementando la validación estricta realizada en el backend mediante middleware `auth:sanctum`.

```
import MisEquiposPage from "../pages/dashboard/MisEquiposPage";
export default function AppRouter() {
  return (
    <BrowserRouter>
      <Routes>
        { /* Público */ }
        <Route path="/login" element={<Login />} />

        { /* Zona protegida */ }
        <Route
          path="/dashboard"
          element={
            <RutaPrivada>
              <AppLayout />
            </RutaPrivada>
          }
        />
        { /* /dashboard */ }
      </Routes>
    </BrowserRouter>
  );
}
```

3. Decisiones Técnicas

3.1 Justificación de Librerías

- **React:** Permite crear una SPA moderna y modular.
- **React Router DOM:** Gestión declarativa del enrutado.

- **Bootstrap:** Diseño responsive rápido y consistente.
- **Laravel Sanctum:** Autenticación segura basada en tokens.
- **Fetch API:** Simplicidad para llamadas HTTP.
- **Vite:** Herramienta de desarrollo y empaquetado que optimiza tiempos de compilación y generación de builds para producción.

Durante el desarrollo se priorizó una arquitectura ligera y controlada, evitando la incorporación de librerías externas innecesarias. Cada dependencia añadida al proyecto fue previamente evaluada en términos de utilidad real, impacto en rendimiento y complejidad estructural. Esta decisión permitió mantener el código más limpio, entendible y mantenible, reduciendo riesgos asociados a dependencias externas y favoreciendo la claridad arquitectónica del sistema.

3.2 Patrón de Diseño Aplicado

En el frontend se aplica:

- Arquitectura modular orientada a dominio funcional.
- Separación entre capa visual y capa de servicios (core/Negocio.js).
- Patrón de contenedor presentación en componentes complejos.

En componentes de mayor complejidad se ha aplicado el patrón contenedor-presentación, separando la lógica de obtención y gestión de datos de la representación visual. Los componentes contenedor gestionan estado y llamadas a la API, mientras que los componentes de presentación se encargan exclusivamente del renderizado de la interfaz. Esta separación mejora la mantenibilidad y reutilización del código.

En el backend se aplica arquitectura en capas (Controller => Service => Repository).

3.3 Gestión de Llamadas al Backend

Las llamadas a la API se encuentran centralizadas en el módulo `src/core/Negocio.js`, implementado como una función autoinvocada (IIFE).

Esta implementación actúa como una capa de servicios en el frontend, alineándose con principios de arquitectura limpia y separación de responsabilidades.

Esta estructura permite encapsular la URL base del backend y exponer únicamente los métodos públicos necesarios para cada recurso del sistema. Cada función corresponde

a un endpoint específico de la API REST, garantizando organización, coherencia y separación de responsabilidades. Asimismo, todas las peticiones protegidas incluyen automáticamente la cabecera Authorization: Bearer <token>, integrando el sistema de autenticación basado en Laravel Sanctum de forma consistente en toda la aplicación.

```
const API_BASE = "http://127.0.0.1:8000/api/v1";
const PROYECTOS_URL = `${API_BASE}/proyectos`;
const TAREAS_URL = `${API_BASE}/tareas`;
const PRIORIDADES_URL = `${API_BASE}/prioridades`;
const USUARIOS_URL = `${API_BASE}/usuarios`;
const EQUIPOS_URL = `${API_BASE}/equipos`;
const ESTADOS_PROYECTO_URL = `${API_BASE}/estado-proyecto`;
const ROLES_URL = `${API_BASE}/roles-equipo`;
const PERMISOS_URL = `${API_BASE}/usuarios-equipo`;
```

```
function getAuthHeaders() {
  const token = localStorage.getItem("token");
  return {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  };
}
```

```
const Negocio = (function () {
  // Métodos para Login y Logout
  async function logIn(email, password) {
```

3.4 Gestión de Seguridad y Autenticación

La autenticación se implementa mediante Laravel Sanctum.

Flujo:

1. Usuario envía credenciales.
2. Backend valida.
3. Sanctum genera token.
4. Token almacenado en localStorage.
5. Token enviado en cada petición protegida.

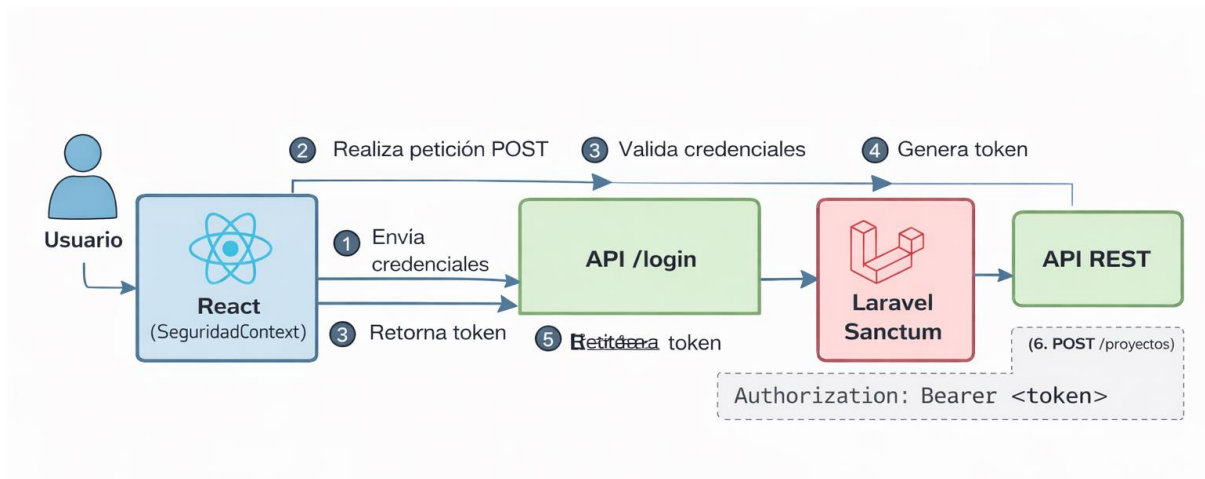
Se implementa middleware auth:sanctum en el backend para proteger rutas sensibles.

El almacenamiento en localStorage se utiliza únicamente para persistencia de sesión en el navegador, sin sustituir la validación de seguridad realizada en el backend.

Sanctum utiliza tokens personales asociados al usuario autenticado, permitiendo control granular sobre el acceso a recursos protegidos.

Esta integración garantiza:

- Seguridad basada en tokens.
- Control de acceso por usuario.
- Protección frente a accesos no autorizados.



4. Despliegue

4.1 Plataforma utilizada

Se ha utilizado Amazon Web Services (AWS):

- EC2 para alojar la aplicación.
- RDS para la base de datos MySQL.
- Nginx como servidor web.
- Certificado SSL para HTTPS.

Esta infraestructura permite escalabilidad horizontal futura y separación entre servidor de aplicación y base de datos

4.2 Proceso de Build

Frontend:

El frontend se construye mediante Vite ejecutando el comando `npm run build`, generando una versión optimizada en la carpeta `/dist`, lista para ser servida por Nginx en entorno de producción.

Backend:

`composer install`

`php artisan config:cache`

`php artisan route:cache`

Esto mejora el rendimiento en entorno productivo.

4.3 URL Pública

La aplicación está accesible en:

<https://taskflow.deviceseru.com>

Repositorio: https://github.com/darioobg/DAW2_GRUPOA_2526

5. Reparto de Tareas

5.1 Organización del Trabajo

El trabajo se organizó por módulos funcionales, manteniendo reuniones periódicas para integración y revisión. Se aplicó una distribución equilibrada por especialización.

5.2 Contribución Individual

Jino Olivera Rudas

- Arquitectura backend.
- Implementación API REST.
- Integración Sanctum.
- Coordinación técnica.

Raúl

- Desarrollo del tablero Kanban.
- CRUD de tareas.
- Gestión dinámica de estados.

Darío

- Diseño del layout general.
- Sidebar y navegación.
- Optimización visual.

María

- Formularios y validaciones.
- Gestión de usuarios y roles.
- Pruebas funcionales y revisión final.
- Despliegue en AWS.